CS402 – High-Performance Computing

Parallel Programming Paradigms

Gihan Mudalige
Professor
Department of Computer Science
University of Warwick
g.mudalige@warwick.ac.uk

A PROBLEM-CENTRIC VIEW OF PARALLELISM

- How do I break the problem apart so multiple compute units can work in parallel ?
  - disregard hardware taxonomy (SIMD,MIMD etc.)
  - disregard programming models taxonomy (i.e. APIs) e.g. OpenMP, MPI etc.

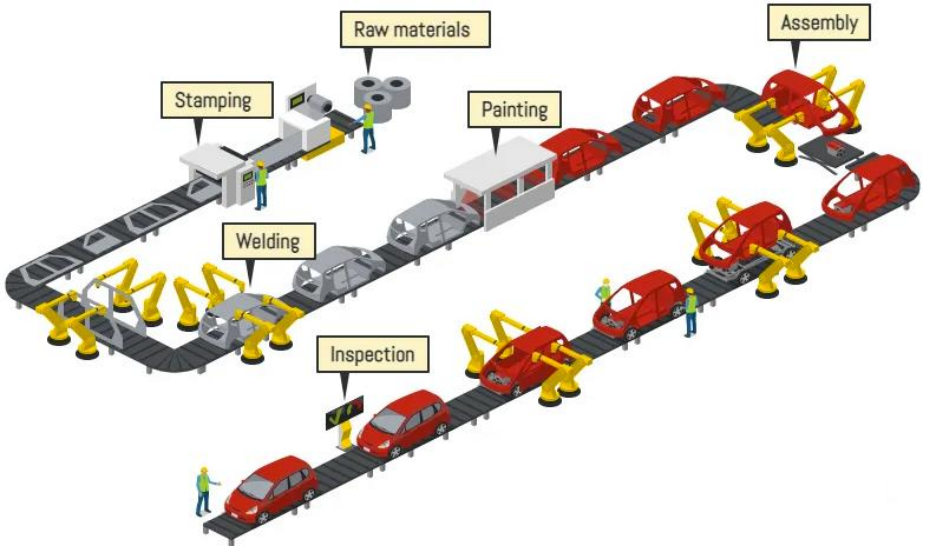
What are the fundamental strategies / structure for Parallelization ?

- There are three fundamental sources/opportunities to exploit
  1. Functional Decomposition/ Functional Parallel – split problem into **different tasks or stages**
  2. Data Decomposition/Data Parallel - apply the **same operation** to different parts of a dataset
  3. Competition - attempt **alternative solutions** in parallel, first success terminates others

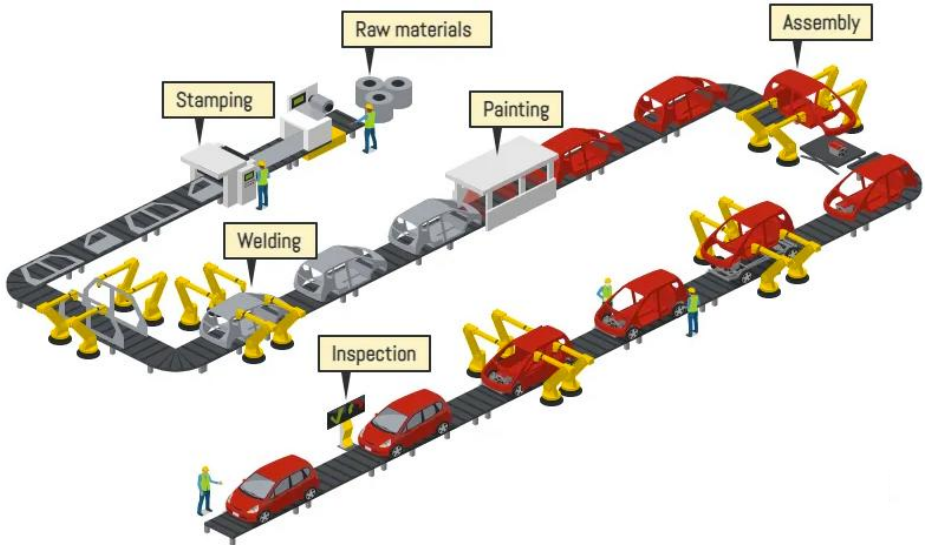
Parallel Programming Paradigms - Functional Decomposition

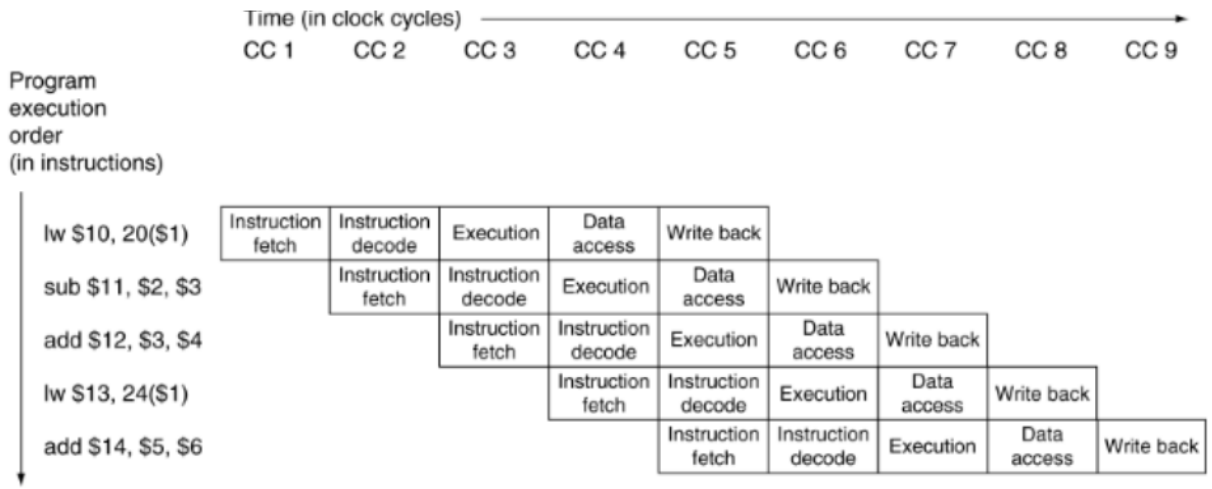
- The problem **decomposes into different steps**
- Different steps that can run in parallel are deployed to **different compute units**

Any gain from parallelism results from different activities that can be completed in parallel.



- **Pipelining –** worker 2 starts painting car 1, while worker 1 starts welding on car 2 .. And so on
  - Need some **warm-up and cool-down phases** : in these not all steps in the pipeline is occupied
  - Highly vulnerable to a time taken by a step compared to others – **slowest step determines the throughput**



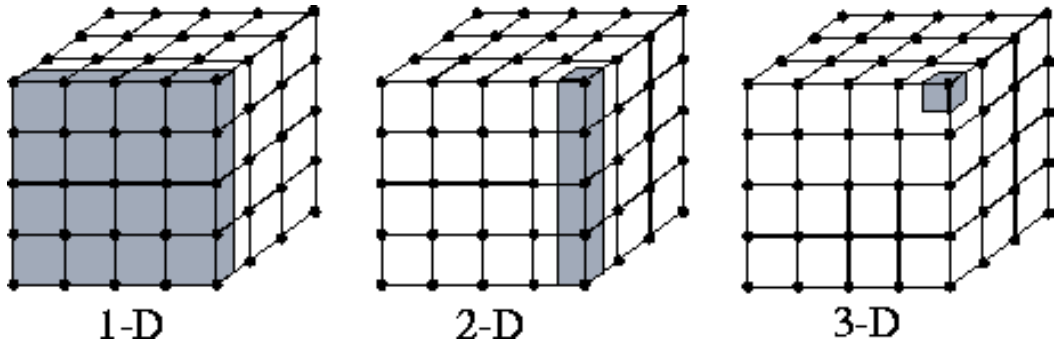


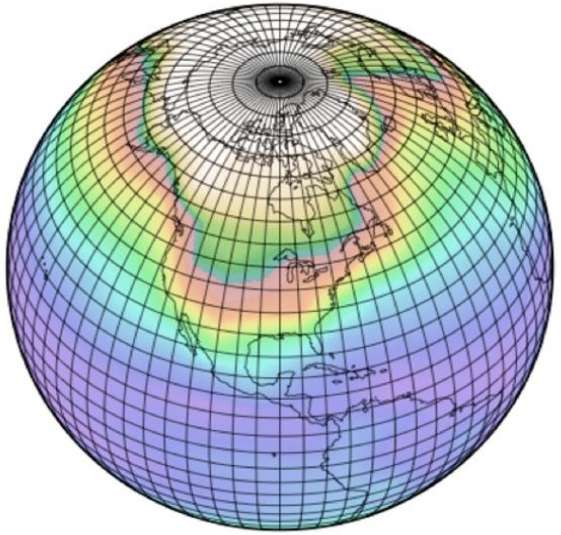
- Pipelining heavily used by computer hardware
  - Bring down the effective cost per instruction to 1 cycle – **one instruction leaves the chip per cycle**
  - Assumes instructions are independent to process independently
  - Multiple instructions done for one piece of data in parallel - **MISD**

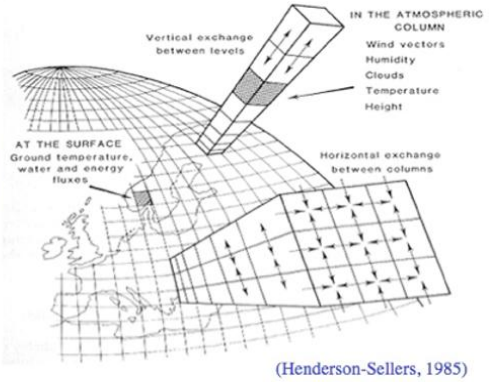
Parallel Programming Paradigms - Data Decomposition

- Task to be done over a data set – **split the data set into smaller chucks (subtasks)**
  - Distribute the sub problem among different compute units, compute in parallel
  - In this module **we will mainly focus on this paradigm**

Performance gain from same activity/compute done over different data in parallel







https://www.nccs.nasa.gov/services/climate-data-services

- SIMD is an extreme case of data parallelism - individual steps per work item run in a synchronized way

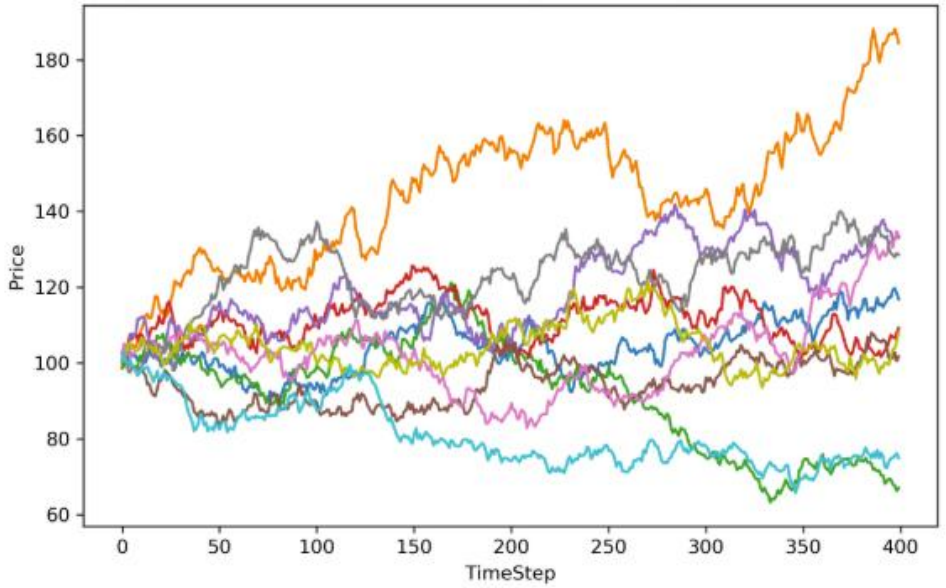
Parallel Programming Paradigms - Competition

- Multiple compute units **work on the same problem at the same time,** then the **first unit to find a valid solution "wins"**
  - Each unit independently explores a different approach, search path or method
  - First one to solution wins, others terminated

Minimize time-to-solution by racing multiple workers against each other.

  - Different units may use : different algorithms, different starting points, different parameter settings
  - Only the winner's results kept, others discorded.

- Examples :
  - Search problems
  - Monte Carlo simulations - Multiple compute units run the *same stochastic algorithm* with **different random seeds**, first unit to reach a sufficiently accurate result terminate the others.

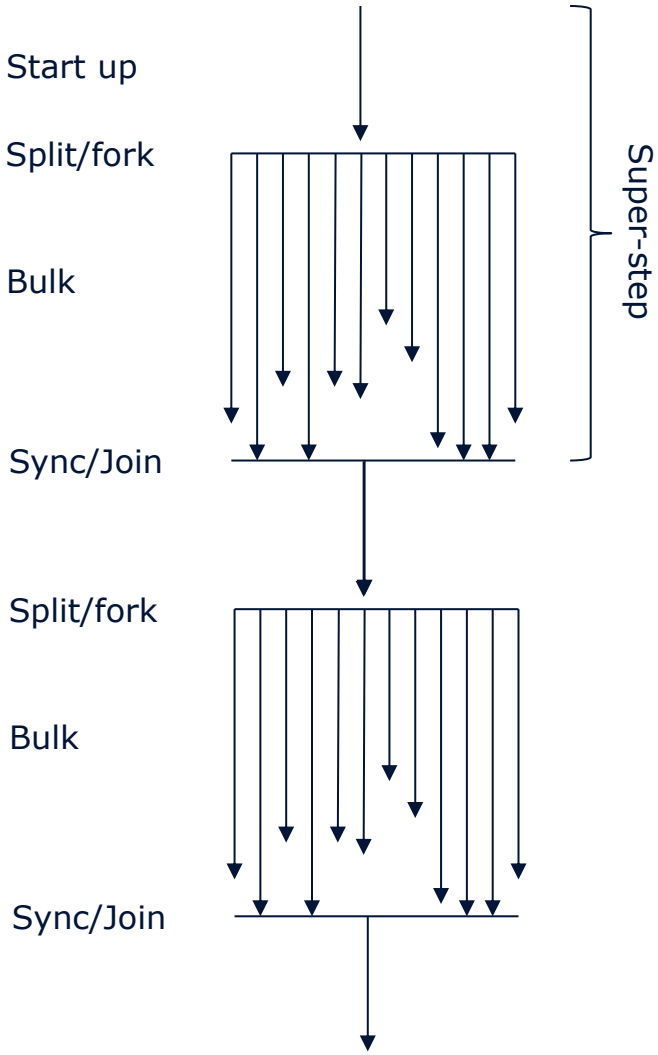


BSP Model – Bulk-Synchronous Processing

- Classical way of **realizing data parallel execution**
  - Proposed by Valiant (Turing Award 2010, Warwick PhD 1974 !)
  - Also provides a cost model
  - BSP is an **execution model**, not a decomposition strategy

- Consists of four conceptual phases
  1. Start-up : serial code
  2. Split code to multiple parts – **Fork**
  3. Compute parts in parallel – **Bulk**
     in original model no communications between parts
  4. Once all parallel activities have terminated,
     fuse the work from each part – **Join/Synchronization**
     some data exchange can occur here

- Multiple Fork-Joins can occur in a program
  - Each called a **super-step** - bulk + synchronisation + final info exchange



SPMD – Single Programme Multiple Data

- BSP is an execution model; **SPMD is the dominant way to *realize* BSP in code**.
  - It is a **program structuring model** that is commonly used to implement BSP-style algorithms.

SPMD – All parallel compute units execute the same program

  - But each will work on different data and may take different execution paths
  - Same code file, same instructions available, but different data per worker + different branches allowed

- In practice, BSP-style algorithms are almost always implemented using SPMD programs.
- BSP and traditional (SISD-focused) vectorization can and are usually realized via SPMD programming.

```
double r[N];
double x[N];
double y[N];
double alpha = ...;

#pragma omp parallel for
for (int i=0; i<N; i++) {
  double alphaX = alpha * x[i];
  r[i] = alphaX + y[i];
}
```

```
int rank;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0) {
    send_data();
} else {
    receive_data();
}
```

```
__global__ void kernel(float *A) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    A[i] = A[i] * 2;
}
```

Contrast with non-SPMD – MPMD
e.g. Client /Server

Parallel Programming Models

- Practical realizations – **programming abstractions/APIs**, that allow programmers to implement in code
  - Paradigms - how is the problem parallelized?
  - Programming models - how do I express this parallelism in code?

We more especially focus on the implementation of BSP programs using SPMD

OpenMP

- **BSP on Multi-cores** (dominant)
  - Written in SPMD style
  - Threads run the same code !
- **Shared-Memory**
- Also can do **functional/task-parallel**

SIMD

- **Special case of BSP**
  - each instruction forms its own super-step
  - Synchronization happens implicitly after the instruction.
- **Register-level vectors**

CUDA/SYCL/HIP

- **SIMD on threads - SIMT**
  - Single Instruction Multiple Threads
  - SIMT- Hardware Execution Model
  - BSP-style structure only at kernel boundaries
- **Device memory + host memory**

MPI

- **BSP on Distributed-memory Procs.**
  - Written in SPMD style
  - Explicit communication and synchronization
  - Processes run the same code !
- **Distributed-Memory**

Further Reading

- Tobias Weinzierl. *Principles of Parallel Scientific Computing, A First Guide to Numerical Concepts and Programming Methods*, Springer (2021). https://link.springer.com/book/10.1007/978-3-030-76194-3 **[Chapter 14]**

- L. G. Valiant: **A bridging model for parallel computation**, *Communications of the ACM*, Volume 33 Issue 8 (1990) https://doi.org/10.1145/79173.79181